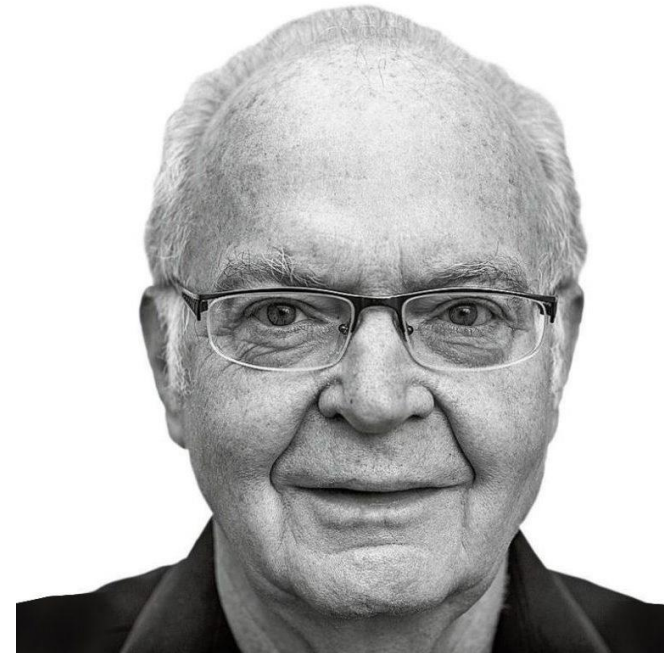


Итеративные сортировки

Горденко Мария Константиновна

Задача сортировки

- Сортировка – это упорядочивание набора однотипных данных по возрастанию или убыванию.



Критерии оценки алгоритмов сортировки

- Время сортировки
- Требуемая память
- Устойчивость

Сфера применения

- Внутренние сортировки
- Внешние сортировки

Стратегии сортировки

- Стратегия выборки
- Стратегия включения
- Стратегия распределения
- Стратегия слияния

Основные идеи итерационных сортировок

- Обмен
- Выбор
- Вставка
- Подсчет

Сортировка выбором (Selection sort)

- Сортировка выбором является одним из простейших алгоритмов сортировки
- Может быть как устойчивой, так и не устойчивой
- Шаги алгоритма:
 - находим минимальное значение в текущей части массива;
 - производим обмен этого значения со значением на первой неотсортированной позиции;
 - далее сортируем хвост массива, исключив из рассмотрения уже отсортированные элементы

Сортировка выбором (Selection sort)

```
function selectionSort(a):  
    for i = 0 to n - 2  
        min = i  
        for j = i + 1 to n - 1  
            if a[j] < a[min]  
                min = j  
        swap(a[i], a[min])
```

- Асимптотическая сложность - $O(n^2)$
- Время работы в худшем, лучшем и среднем случае - $O(n^2)$
- Дополнительная память - $O(1)$

0	1	2	3	4	5	6
7	0	-4	3	1	-2	5

0	1	2	3	4	5	6
7	0	-4	3	1	-2	5

0	1	2	3	4	5	6
7	0	-4	3	1	-2	5

0	1	2	3	4	5	6
7	0	-4	3	1	-2	5

0	1	2	3	4	5	6
7	0	-4	3	1	-2	5

0	1	2	3	4	5	6
7	0	-4	3	1	-2	5

0	1	2	3	4	5	6
7	0	-4	3	1	-2	5

0	1	2	3	4	5	6
7	0	-4	3	1	-2	5

0	1	2	3	4	5	6
7	0	-4	3	1	-2	5

0	1	2	3	4	5	6
7	0	-4	3	1	-2	5

0	1	2	3	4	5	6
7	0	-4	3	1	-2	5

0	1	2	3	4	5	6
7	0	-4	3	1	-2	5

0	1	2	3	4	5	6
-4	0	7	3	1	-2	5

0	1	2	3	4	5	6
-4	0	7	3	1	-2	5

0	1	2	3	4	5	6
-4	0	7	3	1	-2	5

0	1	2	3	4	5	6
-4	0	7	3	1	-2	5

0	1	2	3	4	5	6
-4	0	7	3	1	-2	5

0	1	2	3	4	5	6
-4	0	7	3	1	-2	5

0	1	2	3	4	5	6
-4	0	7	3	1	-2	5

0	1	2	3	4	5	6
-4	0	7	3	1	-2	5

0	1	2	3	4	5	6
-4	0	7	3	1	-2	5

0	1	2	3	4	5	6
-4	0	7	3	1	-2	5

0	1	2	3	4	5	6
-4	0	7	3	1	-2	5

0	1	2	3	4	5	6
-4	-2	7	3	1	0	5

0	1	2	3	4	5	6
-4	-2	7	3	1	0	5

0	1	2	3	4	5	6
-4	-2	7	3	1	0	5

0	1	2	3	4	5	6
-4	-2	7	3	1	0	5

0	1	2	3	4	5	6
-4	-2	7	3	1	0	5

0	1	2	3	4	5	6
-4	-2	7	3	1	0	5

0	1	2	3	4	5	6
-4	-2	7	3	1	0	5

0	1	2	3	4	5	6
-4	-2	7	3	1	0	5

0	1	2	3	4	5	6
-4	-2	7	3	1	0	5

0	1	2	3	4	5	6
-4	-2	7	3	1	0	5

0	1	2	3	4	5	6
-4	-2	7	3	1	0	5

0	1	2	3	4	5	6
-4	-2	7	3	1	0	5

0	1	2	3	4	5	6
-4	-2	0	3	1	7	5

0	1	2	3	4	5	6
-4	-2	0	3	1	7	5

0	1	2	3	4	5	6
-4	-2	0	3	1	7	5

0	1	2	3	4	5	6
-4	-2	0	3	1	7	5

0	1	2	3	4	5	6
-4	-2	0	3	1	7	5

0	1	2	3	4	5	6
-4	-2	0	3	1	7	5

0	1	2	3	4	5	6
-4	-2	0	3	1	7	5

0	1	2	3	4	5	6
-4	-2	0	3	1	7	5

0	1	2	3	4	5	6
-4	-2	0	3	1	7	5

0	1	2	3	4	5	6
-4	-2	0	1	3	7	5

0	1	2	3	4	5	6
-4	-2	0	1	3	7	5

0	1	2	3	4	5	6
-4	-2	0	1	3	7	5

0	1	2	3	4	5	6
-4	-2	0	1	3	7	5

0	1	2	3	4	5	6
-4	-2	0	1	3	7	5

0	1	2	3	4	5	6
-4	-2	0	1	3	7	5

0	1	2	3	4	5	6
-4	-2	0	1	3	7	5

0	1	2	3	4	5	6
-4	-2	0	1	3	7	5

0	1	2	3	4	5	6
-4	-2	0	1	3	7	5

0	1	2	3	4	5	6
-4	-2	0	1	3	7	5

0	1	2	3	4	5	6
-4	-2	0	1	3	7	5

0	1	2	3	4	5	6
-4	-2	0	1	3	7	5

0	1	2	3	4	5	6
-4	-2	0	1	3	5	7

0	1	2	3	4	5	6
-4	-2	0	1	3	5	7

0	1	2	3	4	5	6
-4	-2	0	1	3	5	7

0	1	2	3	4	5	6
-4	-2	0	1	3	5	7

0	1	2	3	4	5	6
-4	-2	0	1	3	5	7

Анализ асимптотики сортировки выбором

- На первой итерации внешнего цикла внутренний цикл сделает $n - 1$ итерацию, на второй — $n - 2$, на последней — 1 итерацию.
- $\sum_{i=0}^{n-2} n - i - 1 = \frac{(n-1)n}{2} = O(n^2)$
- Можно заметить, что количество итераций никак не зависит от элементов массива, а только от его размера, поэтому время работы в лучшем, среднем и худшем случае одинаково.

Сортировка выбором. Достоинства и недостатки

- делает минимальное количество обменов элементов.
- это устойчивый алгоритм сортировки (не меняет порядок элементов, которые уже отсортированы).
- не требует временной памяти, даже под стек.
- **Минусом** же является высокая сложность алгоритма: $O(n^2)$.

Сортировка вставками (Insertion sort)

- Сортировка вставками — алгоритм сортировки, в котором элементы входной последовательности просматриваются по одному, и каждый новый поступивший элемент размещается в подходящее место среди ранее упорядоченных элементов

Сортировка вставками (Insertion sort)

```
function insertionSort(a):  
    for i = 1 to n - 1  
        key = a[i]  
        j = i - 1  
        while j >= 0 and a[j] > key  
            a[j + 1] = a[j]  
            j = j - 1  
        a[j + 1] = key
```

- Асимптотическая сложность - $O(n^2)$
- Время работы в худшем и среднем случае - $O(n^2)$
- Время работы в лучшем случае - $O(n)$
- Дополнительная память - $O(1)$

0	1	2	3	4	5	6
7	0	-4	3	1	-2	5

0	1	2	3	4	5	6
7	0	-4	3	1	-2	5

0	1	2	3	4	5	6
7	0	-4	3	1	-2	5

0	1	2	3	4	5	6
7	0	-4	3	1	-2	5

0	1	2	3	4	5	6
7	0	-4	3	1	-2	5

0	1	2	3	4	5	6
7	0	-4	3	1	-2	5

0	1	2	3	4	5	6
0	7	-4	3	1	-2	5

0	1	2	3	4	5	6
0	7	-4	3	1	-2	5

0	1	2	3	4	5	6
0	7	-4	3	1	-2	5

0	1	2	3	4	5	6
0	7	-4	3	1	-2	5

0	1	2	3	4	5	6
0	7	-4	3	1	-2	5

0	1	2	3	4	5	6
0	7	-4	3	1	-2	5

0	1	2	3	4	5	6
0	-4	7	3	1	-2	5

0	1	2	3	4	5	6
0	-4	7	3	1	-2	5

0	1	2	3	4	5	6
0	-4	7	3	1	-2	5

0	1	2	3	4	5	6
0	-4	7	3	1	-2	5

0	1	2	3	4	5	6
-4	0	7	3	1	-2	5

0	1	2	3	4	5	6
-4	0	7	3	1	-2	5

0	1	2	3	4	5	6
-4	0	7	3	1	-2	5

0	1	2	3	4	5	6
-4	0	7	3	1	-2	5

0	1	2	3	4	5	6
-4	0	7	3	1	-2	5

0	1	2	3	4	5	6
-4	0	7	3	1	-2	5

0	1	2	3	4	5	6
-4	0	3	7	1	-2	5

0	1	2	3	4	5	6
-4	0	3	7	1	-2	5

0	1	2	3	4	5	6
-4	0	3	7	1	-2	5

0	1	2	3	4	5	6
-4	0	3	7	1	-2	5

0	1	2	3	4	5	6
-4	0	3	7	1	-2	5

0	1	2	3	4	5	6
-4	0	3	7	1	-2	5

0	1	2	3	4	5	6
-4	0	3	7	1	-2	5

0	1	2	3	4	5	6
-4	0	3	7	1	-2	5

0	1	2	3	4	5	6
-4	0	3	1	7	-2	5

0	1	2	3	4	5	6
-4	0	3	1	7	-2	5

0	1	2	3	4	5	6
-4	0	3	1	7	-2	5

0	1	2	3	4	5	6
-4	0	3	1	7	-2	5

0	1	2	3	4	5	6
-4	0	1	3	7	-2	5

0	1	2	3	4	5	6
-4	0	1	3	7	-2	5

0	1	2	3	4	5	6
-4	0	1	3	7	-2	5

0	1	2	3	4	5	6
-4	0	1	3	7	-2	5

0	1	2	3	4	5	6
-4	0	1	3	7	-2	5

0	1	2	3	4	5	6
-4	0	1	3	7	-2	5

0	1	2	3	4	5	6
-4	0	1	3	7	-2	5

0	1	2	3	4	5	6
-4	0	1	3	7	-2	5

0	1	2	3	4	5	6
-4	0	1	3	-2	7	5

0	1	2	3	4	5	6
-4	0	1	3	-2	7	5

0	1	2	3	4	5	6
-4	0	1	3	-2	7	5

0	1	2	3	4	5	6
-4	0	1	3	-2	7	5

0	1	2	3	4	5	6
-4	0	1	-2	3	7	5

0	1	2	3	4	5	6
-4	0	1	-2	3	7	5

0	1	2	3	4	5	6
-4	0	1	-2	3	7	5

0	1	2	3	4	5	6
-4	0	1	-2	3	7	5

0	1	2	3	4	5	6
-4	0	-2	1	3	7	5

0	1	2	3	4	5	6
-4	0	-2	1	3	7	5

0	1	2	3	4	5	6
-4	0	-2	1	3	7	5

0	1	2	3	4	5	6
-4	0	-2	1	3	7	5

0	1	2	3	4	5	6
-4	-2	0	1	3	7	5

0	1	2	3	4	5	6
-4	-2	0	1	3	7	5

0	1	2	3	4	5	6
-4	-2	0	1	3	7	5

0	1	2	3	4	5	6
-4	-2	0	1	3	7	5

0	1	2	3	4	5	6
-4	-2	0	1	3	7	5

0	1	2	3	4	5	6
-4	-2	0	1	3	7	5

0	1	2	3	4	5	6
-4	-2	0	1	3	7	5

0	1	2	3	4	5	6
-4	-2	0	1	3	5	7

0	1	2	3	4	5	6
-4	-2	0	1	3	5	7

0	1	2	3	4	5	6
-4	-2	0	1	3	5	7

0	1	2	3	4	5	6
-4	-2	0	1	3	5	7

0	1	2	3	4	5	6
-4	-2	0	1	3	5	7

Анализ асимптотики сортировки вставками

- Если посмотреть на внутренний цикл while, то видно, что наибольшее количество операций выполняется в случае, если добавляемый элемент меньше всех элементов, содержащихся в уже отсортированной части массива. Поэтому наибольшее количество работы алгоритм произведет, когда всякий новый элемент добавляется в начало списка. Такое возможно только, если элементы исходного списка отсортированы в обратном порядке.
- В таком случае на первой итерации внешнего цикла мы поменяем местами **одну** пару элементов, на второй – **две**, на i -ой – i .
- $$\sum_{i=1}^{n-1} i = \frac{(n-1)n}{2} = O(n^2)$$
- В лучшем же случае наш массив уже отсортирован, внутренний while ни разу не отработает, время работы – $O(n)$.

Сортировка бинарными вставками

- Мы можем использовать бинарный поиск, чтобы уменьшить количество сравнений в обычной сортировке вставками. Бинарная сортировка вставками использует бинарный поиск, чтобы найти правильное место для вставки выбранного элемента на каждой итерации.
- При обычной сортировке вставками в худшем случае требуется $O(n)$ сравнений (на n -й итерации). Мы можем уменьшить это до $O(\log n)$, используя бинарный поиск.

Сортировка бинарными вставками

```
function insertionSort(a):  
  for i = 1 to n - 1  
    j = binSearch(a, a[i], 0, i)  
    for k = i - 1 downto j  
      swap(a[k], a[k + 1])
```

- Асимптотическая сложность - $O(n^2)$
- Время работы в худшем и среднем случае - $O(n^2)$
- Время работы в лучшем случае - $O(n)$
- Дополнительная память - $O(1)$

Сортировка вставками. Достоинства и недостатки

- эффективен на небольших наборах данных, на наборах данных до десятков элементов может оказаться лучшим;
- эффективен на наборах данных, которые уже частично отсортированы;
- это устойчивый алгоритм сортировки (не меняет порядок элементов, которые уже отсортированы);
- может сортировать список по мере его получения;
- не требует временной памяти, даже под стек.
- **Минусом** же является высокая сложность алгоритма: $O(n^2)$.

Сортировка пузырьком (bubble sort)

- Алгоритм состоит из повторяющихся проходов по сортируемому массиву. За каждый проход элементы последовательно сравниваются попарно и, если порядок в паре неверный, выполняется обмен элементов. Проходы по массиву повторяются $N-1$ раз или до тех пор, пока на очередном проходе не окажется, что обмены больше не нужны, что означает — массив отсортирован. При каждом проходе алгоритма по внутреннему циклу, очередной наибольший элемент массива ставится на своё место в конце массива рядом с предыдущим «наибольшим элементом», а наименьший элемент перемещается на одну позицию к началу массива («всплывает» до нужной позиции как пузырёк в воде, отсюда и название алгоритма).

Сортировка пузырьком (bubble sort)

```
function bubbleSort(a):  
  for i = 0 to n - 2  
    for j = 0 to n - i - 2  
      if a[j] > a[j + 1]  
        swap(a[j], a[j + 1])
```

- Асимптотическая сложность - $O(n^2)$
- Время работы в худшем и среднем случае - $O(n^2)$
- Время работы в лучшем случае - $O(n)$
- Дополнительная память - $O(1)$

0	1	2	3	4	5	6
7	0	-4	3	1	-2	5

0	1	2	3	4	5	6
7	0	-4	3	1	-2	5

0	1	2	3	4	5	6
7	0	-4	3	1	-2	5

0	1	2	3	4	5	6
0	7	-4	3	1	-2	5

0	1	2	3	4	5	6
0	7	-4	3	1	-2	5

0	1	2	3	4	5	6
0	7	-4	3	1	-2	5

0	1	2	3	4	5	6
0	-4	7	3	1	-2	5

0	1	2	3	4	5	6
0	-4	7	3	1	-2	5

0	1	2	3	4	5	6
0	-4	7	3	1	-2	5

0	1	2	3	4	5	6
0	-4	3	7	1	-2	5

0	1	2	3	4	5	6
0	-4	3	7	1	-2	5

0	1	2	3	4	5	6
0	-4	3	7	1	-2	5

0	1	2	3	4	5	6
0	-4	3	1	7	-2	5

0	1	2	3	4	5	6
0	-4	3	1	7	-2	5

0	1	2	3	4	5	6
0	-4	3	1	7	-2	5

0	1	2	3	4	5	6
0	-4	3	1	-2	7	5

0	1	2	3	4	5	6
0	-4	3	1	-2	7	5

0	1	2	3	4	5	6
0	-4	3	1	-2	7	5

0	1	2	3	4	5	6
0	-4	3	1	-2	5	7

0	1	2	3	4	5	6
0	-4	3	1	-2	5	7

0	1	2	3	4	5	6
0	-4	3	1	-2	5	7

0	1	2	3	4	5	6
0	-4	3	1	-2	5	7

0	1	2	3	4	5	6
-4	0	3	1	-2	5	7

0	1	2	3	4	5	6
-4	0	3	1	-2	5	7

0	1	2	3	4	5	6
-4	0	3	1	-2	5	7

0	1	2	3	4	5	6
-4	0	3	1	-2	5	7

0	1	2	3	4	5	6
-4	0	1	3	-2	5	7

0	1	2	3	4	5	6
-4	0	1	3	-2	5	7

0	1	2	3	4	5	6
-4	0	1	3	-2	5	7

0	1	2	3	4	5	6
-4	0	1	-2	3	5	7

0	1	2	3	4	5	6
-4	0	1	-2	3	5	7

0	1	2	3	4	5	6
-4	0	1	-2	3	5	7

0	1	2	3	4	5	6
-4	0	1	-2	3	5	7

0	1	2	3	4	5	6
-4	0	1	-2	3	5	7

0	1	2	3	4	5	6
-4	0	1	-2	3	5	7

0	1	2	3	4	5	6
-4	0	1	-2	3	5	7

0	1	2	3	4	5	6
-4	0	-2	1	3	5	7

0	1	2	3	4	5	6
-4	0	-2	1	3	5	7

0	1	2	3	4	5	6
-4	0	-2	1	3	5	7

0	1	2	3	4	5	6
-4	0	-2	1	3	5	7

0	1	2	3	4	5	6
-4	0	-2	1	3	5	7

0	1	2	3	4	5	6
-4	0	-2	1	3	5	7

0	1	2	3	4	5	6
-4	-2	0	1	3	5	7

0	1	2	3	4	5	6
-4	-2	0	1	3	5	7

0	1	2	3	4	5	6
-4	-2	0	1	3	5	7

0	1	2	3	4	5	6
-4	-2	0	1	3	5	7

0	1	2	3	4	5	6
-4	-2	0	1	3	5	7

0	1	2	3	4	5	6
-4	-2	0	1	3	5	7

0	1	2	3	4	5	6
-4	-2	0	1	3	5	7

0	1	2	3	4	5	6
-4	-2	0	1	3	5	7

0	1	2	3	4	5	6
-4	-2	0	1	3	5	7

Оптимизация (условие Айверсона 1)

- Если после выполнения внутреннего цикла не произошло ни одного обмена, то массив уже отсортирован, и продолжать что-то делать бессмысленно. Поэтому внутренний цикл можно выполнять не $n-1$ раз, а до тех пор, пока во внутреннем цикле происходят обмены.

```
function bubbleSort(a):  
    i = 0  
    t = true  
    while t  
        t = false  
        for j = 0 to n - i - 2  
            if a[j] > a[j + 1]  
                swap(a[j], a[j + 1])  
                t = true  
        i = i + 1
```

Оптимизация (условие Айверсона 2)

- Запоминаем, в какой позиции t был последний обмен на предыдущей итерации внешнего цикла.
- Это – верхняя граница просмотра массива на следующей итерации
- Если $t = 0$ после выполнения внутреннего цикла, значит, обменов не было, алгоритм заканчивает работу.
- Основная идея – **уменьшаем количество проходов внутреннего цикла**

```
function bubbleSort(a):  
    t = n - 1  
    while t > 0  
        m = t  
        t = 0  
        for i = 0 to m  
            if a[i] > a[i + 1]  
                swap(a[i], a[i + 1])  
            t = i
```

Анализ асимптотики сортировки пузырьком

- Худшим случаем является массив отсортированный в обратном порядке. Наибольший элемент, что стоит первым, будет переставляться со всеми остальными элементами вплоть до конца списка. Второй наибольший элемент, будет переставляться вплоть до предпоследнего места. Этот процесс повторяется для всех остальных элементов, поэтому внешний цикл `while` отработает $n - 1$ раз.
- $$\sum_{i=1}^{n-1} i = \frac{(n-1)n}{2} = O(n^2)$$
- В лучшем же случае, наш массив уже отсортирован, если мы применяем оптимизации, внешний цикл `while` отработает один раз, время работы – $O(n)$.

Сортировка пузырьком. Достоинства и недостатки

- это устойчивый алгоритм сортировки (не меняет порядок элементов, которые уже отсортированы);
- не требует временной памяти, даже под стек.
- **Минусом** же является высокая сложность алгоритма: $O(n^2)$.

Модификации

- Сортировка чет-нечет
- Сортировка расческой
- Сортировка перемешиванием (шейкерная сортировка)

Линейные сортировки

Горденко Мария Константиновна

Сортировка подсчетом (Counting sort)

- Алгоритм сортировки, в котором используется диапазон чисел сортируемого массива (списка) для подсчёта совпадающих элементов
- Применение сортировки подсчётом целесообразно лишь тогда, когда сортируемые числа имеют (или их можно отобразить в) диапазон возможных значений, который достаточно мал по сравнению с сортируемым множеством
- Есть устойчивый и неустойчивый вариант сортировки

Сортировка подсчетом неустойчивая

```
1. SimpleCountingSort:
2.   for i = 0 to k - 1
3.     C[i] = 0;
4.   for i = 0 to n - 1
5.     C[A[i]] = C[A[i]] + 1;
6.   b = 0;
7.   for j = 0 to k - 1
8.     for i = 0 to C[j] - 1
9.       A[b] = j;
10.      b = b + 1;
```

k – количество различных элементов массива

n – число элементов массива A

В приведённом псевдокоде считается, что минимальный элемент массива – 0, но можно обобщить данный алгоритм и на случай отрицательных чисел

A

0	1	2	3	4	5	6	7	8
3	4	1	0	1	2	2	4	4

C

0	1	2	3	4
1	2	2	1	3

A

j=0; i=0

0	1	2	3	4	5	6	7	8
0	4	1	0	1	2	2	4	4

j=1; i=0..1

0	1	2	3	4	5	6	7	8
0	1	1	0	1	2	2	4	4

j=2; i=0..1

0	1	2	3	4	5	6	7	8
0	1	1	2	2	2	2	4	4

j=3; i=0

0	1	2	3	4	5	6	7	8
0	1	1	2	2	3	2	4	4

j=4; i=0..2

0	1	2	3	4	5	6	7	8
0	1	1	2	2	3	4	4	4

Сортировка подсчетом устойчивая

```
1. StableCountingSort
2.   for i = 0 to k
3.     C[i] = 0;
4.   for i = 0 to n - 1
5.     C[A[i]] = C[A[i]] + 1;
6.   for j = 1 to k           // находим частичные суммы, число
7.     C[j] = C[j] + C[j - 1]; // элементов не больших j
8.   for i = n - 1 to 0
9.     C[A[i]] = C[A[i]] - 1;
10.    B[C[A[i]]] = A[i];
```

Сортировка подсчетом устойчивая

A

0	1	2	3	4	5	6	7
3	6	4	1	3	4	1	4

C

0	1	2	3	4	5	6
0	2	0	2	3	0	1

for i = 0 to n - 1
C[A[i]] = C[A[i]] + 1

C

0	1	2	3	4	5	6
0	2	2	4	7	7	8

for j = 1 to k
C[j] = C[j] + C[j-1]

i = 7 B[C[A[7]]-1]=B[C[4]-1]=B[6]=A[7]

B

0	1	2	3	4	5	6	7
						4	

C

0	1	2	3	4	5	6
0	2	2	4	6	7	8

for i = n - 1 to 0
B[C[A[i]]-1] = A[i];
C[A[i]] = C[A[i]] - 1

i = 6 B[C[A[6]]-1] = B[C[1]-1] => B[1] = A[6]

B

0	1	2	3	4	5	6	7
	1					4	

C

0	1	2	3	4	5	6
0	1	2	4	6	7	8

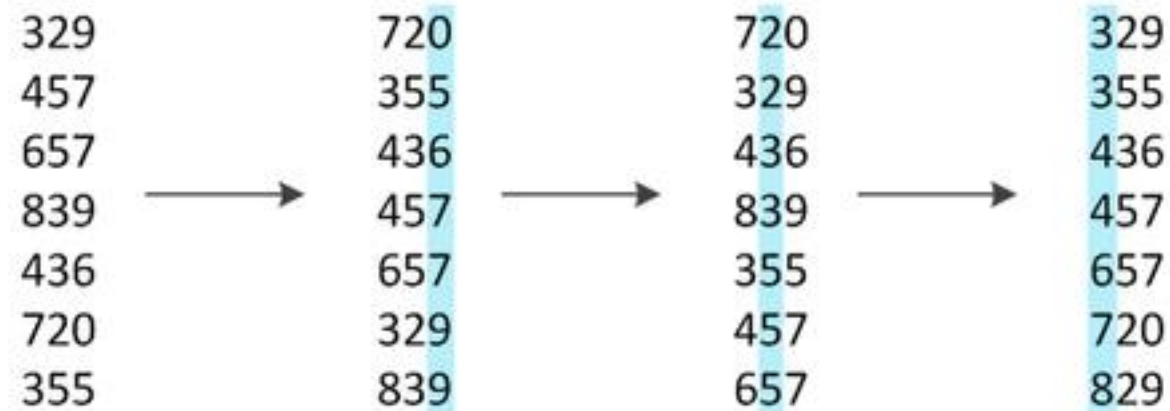
Достоинства и недостатки

- Асимптотическая сложность - $O(n + k)$
- Сортировка выполняется только для целых чисел
- Целесообразно применять, когда диапазон чисел небольшой по сравнению с размером сортируемого массива
- Требуется дополнительная память - $O(k)$ в случае неустойчивой сортировки и $O(n + k)$ в случае устойчивой

Цифровая сортировка (Radix sort)

- Один из алгоритмов сортировки, использующих внутреннюю структуру сортируемых объектов
- Алгоритм пригоден для сортировки любых объектов, запись которых можно поделить на «разряды», содержащие сравнимые значения

LSD (Least Significant Digit radix sort)



- Сортировку по разрядам можно провести подсчетом, тогда цифровая сортировка будет линейной
- Поразрядная сортировка массива будет работать только в том случае, если сортировка, выполняющаяся по разряду, является устойчивой

LSD (Least Significant Digit radix sort)

```
1. function radixSort(int[] A):
2.     for i = 1 to m
3.         for j = 0 to k - 1
4.             C[j] = 0
5.             for j = 0 to n - 1
6.                 d = digit(A[j], i)
7.                 C[d]++
8.             count = 0
9.             for j = 0 to k - 1
10.                tmp = C[j]
11.                C[j] = count
12.                count += tmp
13.             for j = 0 to n - 1
14.                 d = digit(A[j], i)
15.                 B[C[d]] = A[j]
16.                 C[d]++
17.             A = B
```

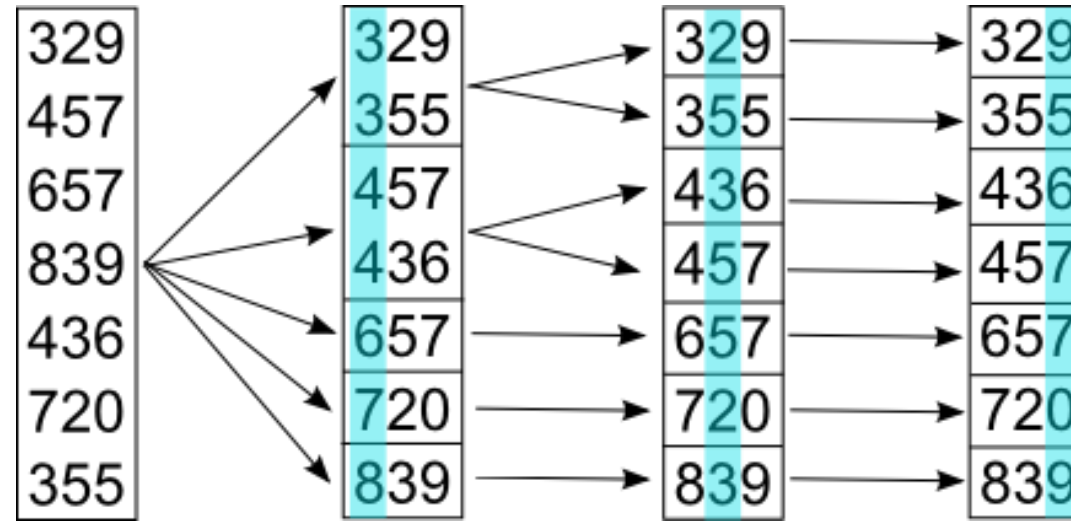
Сложность LSD-сортировки

Пусть m — количество разрядов, n — количество объектов, которые нужно отсортировать, $T(n)$ — время работы устойчивой сортировки. Цифровая сортировка выполняет m итераций, на каждой из которой выполняется устойчивая сортировка и не более $O(1)$ других операций. Следовательно время работы цифровой сортировки — $O(mT(n))$.

Рассмотрим отдельно случай сортировки чисел. Пусть в качестве аргумента сортировке передается массив, в котором содержатся n m -значных чисел, и каждая цифра может принимать значения от 0 до $k - 1$. Тогда цифровая сортировка позволяет отсортировать данный массив за время $O(m(n + k))$, если устойчивая сортировка имеет время работы $O(n + k)$. Если k небольшое, то оптимально выбирать в качестве устойчивой сортировки сортировку подсчетом.

Если количество разрядов — константа, а $k = O(n)$, то сложность цифровой сортировки составляет $O(n)$, то есть она линейно зависит от количества сортируемых чисел.

MSD (Most Significant Digit radix sort)



MSD (Most Significant Digit radix sort)

```
1. function radixSort(int[] A, int l, int r, int d):
2.     if d > m or l >= r
3.         return
4.     for j = 0 to k + 1
5.         cnt[j] = 0
6.     for i = l to r
7.         j = digit(A[i], d)
8.         cnt[j + 1]++
9.     for j = 2 to k
10.        cnt[j] += cnt[j - 1]
11.    for i = l to r
12.        j = digit(A[i], d)
13.        c[l + cnt[j]] = A[i]
14.        cnt[j]--
15.    for i = l to r
16.        A[i] = c[i]
17.    radixSort(A, l, l + cnt[0] - 1, d + 1)
18.    for i = 1 to k
19.        radixSort(A, l + cnt[i - 1], l + cnt[i] - 1, d + 1)
```

Сложность MSD-сортировки

Пусть значения разрядов меньше b , а количество разрядов — k . При сортировке массива из одинаковых элементов MSD-сортировкой на каждом шаге все элементы будут находиться в неубывающей по размеру корзине, а так как цикл идет по всем элементам массива, то получим, что время работы MSD-сортировки оценивается величиной $O(nk)$, причем это время нельзя улучшить. Хорошим случаем для данной сортировки будет массив, при котором на каждом шаге каждая корзина будет делиться на b частей. Как только размер корзины станет равен 1, сортировка перестанет рекурсивно запускаться в этой корзине. Таким образом, асимптотика будет $\Omega(n \log_b n)$. Это хорошо тем, что не зависит от числа разрядов.

Существует также модификация MSD-сортировки, при которой рекурсивный процесс останавливается при небольших размерах текущего кармана, и вызывается более быстрая сортировка, основанная на сравнениях (например, сортировка вставками).

Как выделять цифры числа?

- `digit(A[i], d)`
- Можно выделять цифры в десятичной системе счисления
- Можно учесть формат хранения данных (двоичная система счисления)
- Пусть, сортируются 4-байтовые без знаковые числа, значения которых лежат в интервале $[0, 32767]$

Основание СС	Размер массива С	Количество цифр числа	Выделение цифр
10			
2			
16			
256			

Цифровая сортировка по основанию 256

Сортировка подсчётом по i -му байту будет проходить в два прохода по исходному массиву:

- при первом проходе по исходному массиву выполняется подсчёт i -ых байт в массиве **mas**, результат будет сохранён в массив подсчётов **counter** из 256 элементов;
- в массив **offset** на основании посчитанных данных выполняется подсчёт смещений, по которым будут сохраняться элементы:

```
offset[0]=0
```

для всех j от 1 до 255

```
offset[j]=counter[j-1]+offset[j-1]
```

- при втором проходе по исходному массиву **mas** выполняется копирование элемента во вспомогательный массив по соответствующему индексу в массиве смещений **offset** и выполняется инкремент смещения.

Цифровая сортировка по основанию 256

Модифицированная реализация сортировки подсчётом требует 1 КБ дополнительной памяти и имеет out-of-place реализацию.

Целочисленные типы без знака представляются в памяти так (рассмотрим 4-байтовый тип **unsigned int**), что число x кодируется следующим образом:

$$x = \sum_{i=0}^{31} 2^i \cdot b_i, \text{ где } b_i - i\text{-ый бит числа.}$$

Поразрядная сортировка таких чисел будет выполняться в 4 прохода, начиная от младшего байта до старшего.

Цифровая сортировка знаковых чисел

Целочисленный тип со знаком кодируется таким образом, что при сложении равных по модулю отрицательного и положительного чисел получается нуль (так называемый дополнительный код). Кроме того, старший бит определяет знак числа:

- если старший бит равен нулю, то число положительное;
- если старший бит равен единице, то число отрицательное.

Цифровая сортировка знаковых чисел

Целочисленные типы со знаком представляются в памяти так (рассмотрим 4-байтовый тип **signed int**), что число x кодируется следующим образом:

$$x = \sum_{i=0}^{30} 2^i \cdot b_i - 2^{31} \cdot b_{31} \quad , \text{ где } b_i - i\text{-ый бит числа.}$$

Старший бит знаковых типов определяет знак числа, поэтому сортировку нужно выполнять с учётом того, что числа? у которых старший бит равен 0 (положительные) больше тех чисел у которых старший бит равен 1 (отрицательные). Исходя из этого, получается, что при сортировке знаковых чисел необходимо изменить алгоритм сортировки только по старшему байту. Для этого достаточно в сортировке подсчётом интерпретировать старший байт, как знаковое число (диапазон знаковых однобайтовых чисел от **-128** до **127**) и прибавлять смещение **128**. Таким образом, в массиве подсчётов по индексу 0 будет находиться количество элементов массива, у которых в старшем байте минимальное число со знаком (**-128**).